12.2 TypeBox 类型模式（Schema）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~200,000 tokens，输出 ~15,000 tokens（本章合计）

上一节里，我们看到了协议帧的 Schema 定义，比如 `RequestFrameSchema` 、 `EventFrameSchema` 等。你可能注意到了这些定义使用了 `Type.Object()` 、 `Type.String()` 等语法——这正是 TypeBox 库提供的 API。本节深入分析 OpenClaw 如何利用 TypeBox 构建一套"定义一次、多端共用"的协议类型系统。

12.2.1 协议 Schema 定义

什么是 TypeBox?

> **衍生解释**：TypeBox 是一个 TypeScript 库（ `@sinclair/typebox` ），它的核心理念是：**用 TypeScript 代码定义 JSON Schema，同时自动获得对应的 TypeScript 类型**。
>
> 在传统开发中，你通常需要分别维护两份定义：
>
> - 一份 TypeScript 接口，用于编译时类型检查
> - 一份 JSON Schema 或 Zod Schema，用于运行时数据校验
>
> 这两份定义很容易不一致。TypeBox 通过巧妙的类型体操（Type Gymnastics），让你写一份代码就能同时得到两者。

来看一个简单的例子：

```typescript
import { Type, type Static } from "@sinclair/typebox";

// 定义 Schema（既是 JSON Schema 的值，也是 TypeScript 类型的来源）
const UserSchema = Type.Object({
  name: Type.String({ minLength: 1 }),
  age: Type.Integer({ minimum: 0 }),
  email: Type.Optional(Type.String({ format: "email" })),
});

// 从 Schema 中提取 TypeScript 类型
type User = Static<typeof UserSchema>;
// 等价于：
// type User = {
//   name: string;
//   age: number;
//   email?: string;
// }
```

在运行时， `UserSchema` 的值就是一个标准的 JSON Schema 对象：

```json
{
  "type": "object",
  "properties": {
    "name": { "type": "string", "minLength": 1 },
    "age": { "type": "integer", "minimum": 0 },
    "email": { "type": "string", "format": "email" }
  },
  "required": ["name", "age"]
}
```

OpenClaw 的 Schema 组织结构

OpenClaw 的协议 Schema 定义在 `src/gateway/protocol/schema/` 目录下，按功能领域划分为多个文件：

```
src/gateway/protocol/schema/
├── primitives.ts          # 基础类型（NonEmptyString 等）
├── frames.ts              # 帧类型（RequestFrame、ResponseFrame、EventFrame）
├── agent.ts               # Agent 相关（AgentParams、SendParams 等）
├── sessions.ts            # 会话管理（SessionsList、SessionsPatch 等）
├── snapshot.ts            # 状态快照（Presence、Health、StateVersion）
├── config.ts              # 配置操作（ConfigGet、ConfigSet 等）
├── channels.ts            # 通道操作（ChannelsStatus、TalkMode 等）
├── devices.ts             # 设备配对（DevicePair 相关）
├── nodes.ts               # 节点操作（NodePair、NodeInvoke 等）
├── cron.ts                # 定时任务（CronAdd、CronRemove 等）
├── exec-approvals.ts      # 执行审批
├── agents-models-skills.ts # Agent/模型/技能列表
├── logs-chat.ts           # 日志与 WebChat
├── wizard.ts              # 引导向导
├── error-codes.ts         # 错误码定义
├── types.ts               # 类型导出（Static<> 提取）
└── protocol-schemas.ts    # Schema 注册表
```

所有文件通过 `schema.ts` 这个桶文件（Barrel File）统一导出：

```python
// src/gateway/protocol/schema.ts
export * from "./schema/agent.js";
export * from "./schema/agents-models-skills.js";
export * from "./schema/channels.js";
export * from "./schema/config.js";
// ...共 16 个导出
export * from "./schema/wizard.js";
```

基础类型（Primitives）

所有 Schema 都建立在几个基础类型之上：

```
// src/gateway/protocol/schema/primitives.ts
import { Type } from "@sinclair/typebox";
import { SESSION_LABEL_MAX_LENGTH } from "../../../sessions/session-label.js";
import { GATEWAY_CLIENT_IDS, GATEWAY_CLIENT_MODES } from "../client-info.js";

// 非空字符串 —— 协议中最常用的基础类型
export const NonEmptyString = Type.String({ minLength: 1 });

// 会话标签字符串（有最大长度限制）
export const SessionLabelString = Type.String({
  minLength: 1,
  maxLength: SESSION_LABEL_MAX_LENGTH,
});

// 客户端 ID 枚举（通过 Type.Union + Type.Literal 构建）
export const GatewayClientIdSchema = Type.Union(
  Object.values(GATEWAY_CLIENT_IDS).map((value) => Type.Literal(value)),
);

// 客户端模式枚举
export const GatewayClientModeSchema = Type.Union(
  Object.values(GATEWAY_CLIENT_MODES).map((value) => Type.Literal(value)),
);
```

这里有一个巧妙的设计：`GatewayClientIdSchema` 不是手动列出所有可能的值，而是从常量对象 `GATEWAY_CLIENT_IDS` 动态生成的。这意味着当你在 `client-info.ts` 中添加一个新的客户端类型时，Schema 会自动包含它，**无需手动同步**。

`GATEWAY_CLIENT_IDS` 定义了 OpenClaw 支持的所有客户端类型：

```typescript
// src/gateway/protocol/client-info.ts
export const GATEWAY_CLIENT_IDS = {
  WEBCHAT_UI: "webchat-ui",        // WebChat UI
  CONTROL_UI: "openclaw-control-ui", // Web 控制台
  WEBCHAT: "webchat",              // 内嵌 WebChat
  CLI: "cli",                      // 命令行工具
  GATEWAY_CLIENT: "gateway-client", // 通用 Gateway 客户端
  MACOS_APP: "openclaw-macos",     // macOS 应用
  IOS_APP: "openclaw-ios",         // iOS 应用
  ANDROID_APP: "openclaw-android", // Android 应用
  NODE_HOST: "node-host",          // 节点宿主
  TEST: "test",                    // 测试客户端
  FINGERPRINT: "fingerprint",      // 指纹识别
  PROBE: "openclaw-probe",         // 探针
} as const;
```

类型提取模式

`types.ts` 文件使用 TypeBox 的 `Static<>` 工具类型，从每个 Schema 中提取出对应的 TypeScript 类型：

```typescript
// src/gateway/protocol/schema/types.ts
import type { Static } from "@sinclair/typebox";
import type { ConnectParamsSchema, HelloOkSchema, /* ... */ } from "./frames.js";

export type ConnectParams = Static<typeof ConnectParamsSchema>;
export type HelloOk = Static<typeof HelloOkSchema>;
export type RequestFrame = Static<typeof RequestFrameSchema>;
export type ResponseFrame = Static<typeof ResponseFrameSchema>;
// ... 共 90+ 个类型导出
```

这种 `Static<typeof XxxSchema>` 的模式在 OpenClaw 中大量使用。它保证了 TypeScript 编译器看到的类型与运行时校验使用的 Schema 始终一致——因为它们本质上是**同一个定义的两个视角**。

运行时校验（AJV 编译）

有了 Schema 定义之后，OpenClaw 使用 AJV（Another JSON Validator）库将它们编译为高效的校验函数：

```
// src/gateway/protocol/index.ts
import AjvPkg from "ajv";

const ajv = new AjvPkg({
  allErrors: true,     // 收集所有错误，而不是遇到第一个就停
  strict: false,       // 关闭严格模式（兼容 TypeBox 的输出）
  removeAdditional: false, // 不自动移除多余字段
});

// 为每个 Schema 编译出一个校验函数
export const validateConnectParams = ajv.compile<ConnectParams>(ConnectParamsSchema);
export const validateRequestFrame = ajv.compile<RequestFrame>(RequestFrameSchema);
export const validateResponseFrame = ajv.compile<ResponseFrame>(ResponseFrameSchema);
export const validateEventFrame = ajv.compile<EventFrame>(EventFrameSchema);
// ... 共 70+ 个校验函数
```

> **衍生解释**：AJV（Another JSON Validator）是目前 JavaScript 生态中性能最好的 JSON Schema 校验库。它的核心优化策略是**预编译**——调用 `ajv.compile(schema)` 时，AJV 会将 JSON Schema 编译成一段优化过的 JavaScript 代码，后续的校验调用几乎就是直接执行这段代码，速度极快。这与正则表达式的"先编译后匹配"理念类似。

这些编译后的校验函数在 Gateway 中被用于：

1. **入站消息校验**：WebSocket 收到消息后，先校验帧格式是否合法
2. **方法参数校验**：对于每个方法调用，校验 `params` 是否符合预期格式
3. **出站消息校验**：（可选）在开发模式下校验服务器发出的消息

校验错误格式化

当校验失败时，AJV 会返回一个 `ErrorObject[]` 数组。OpenClaw 提供了一个专门的格式化函数，将它转换为人类可读的错误消息：

```
// src/gateway/protocol/index.ts
export function formatValidationErrors(errors: ErrorObject[] | null | undefined) {
  if (!errors?.length) {
    return "unknown validation error";
  }
  const parts: string[] = [];
  for (const err of errors) {
    if (err.keyword === "additionalProperties") {
      // 特殊处理：告诉用户哪个字段是多余的
      const where = err.instancePath ? `at ${err.instancePath}` : "at root";
      parts.push(`${where}: unexpected property '${additionalProperty}'`);
      continue;
    }
    // 通用处理
    const where = err.instancePath ? `at ${err.instancePath}: ` : "";
    parts.push(`${where}${err.message}`);
  }
  // 去重后拼接
  return Array.from(new Set(parts)).join("; ");
}
```

这个函数对 `additionalProperties` 错误做了特殊处理——不只是说"校验失败"，而是明确告诉开发者"你发送了一个多余的字段 `token` "。这种友好的错误提示对于调试协议交互非常有帮助。

12.2.2 从 TypeBox 生成 JSON Schema

TypeBox 定义本身就是 JSON Schema 的 JavaScript 表示。但 OpenClaw 还需要将它们导出为**独立的 JSON Schema 文件**，原因有二：

1. **给非 TypeScript 客户端使用**：Swift（iOS/macOS）、Kotlin（Android）客户端无法直接使用 TypeBox 定义
2. **协议文档化**：JSON Schema 可以被工具渲染为可读的文档

`scripts/protocol-gen.ts` 脚本负责这个转换：

```typescript
// scripts/protocol-gen.ts
import { ProtocolSchemas } from "../src/gateway/protocol/schema.js";

async function writeJsonSchema() {
  const definitions: Record<string, unknown> = {};
  for (const [name, schema] of Object.entries(ProtocolSchemas)) {
    definitions[name] = schema;
  }

  const rootSchema = {
    $schema: "http://json-schema.org/draft-07/schema#",
    $id: "https://openclaw.ai/protocol.schema.json",
    title: "OpenClaw Gateway Protocol",
    description: "Handshake, request/response, and event frames for the Gateway WebSock
    oneOf: [
      { $ref: "#/definitions/RequestFrame" },
      { $ref: "#/definitions/ResponseFrame" },
      { $ref: "#/definitions/EventFrame" },
    ],
    discriminator: {
      propertyName: "type",
      mapping: {
        req: "#/definitions/RequestFrame",
        res: "#/definitions/ResponseFrame",
        event: "#/definitions/EventFrame",
      },
    },
    definitions,
  };

  const jsonSchemaPath = path.join(repoRoot, "dist", "protocol.schema.json");
  await fs.writeFile(jsonSchemaPath, JSON.stringify(rootSchema, null, 2));
}
```

`ProtocolSchemas` 是一个注册表（Registry），将所有 Schema 汇总到一个 `Record<string, TSchema>` 中：

```typescript
// src/gateway/protocol/schema/protocol-schemas.ts
import type { TSchema } from "@sinclair/typebox";

export const ProtocolSchemas: Record<string, TSchema> = {
  ConnectParams: ConnectParamsSchema,
  HelloOk: HelloOkSchema,
  RequestFrame: RequestFrameSchema,
  ResponseFrame: ResponseFrameSchema,
  EventFrame: EventFrameSchema,
  GatewayFrame: GatewayFrameSchema,
  // ... 共 80+ 个 Schema
  ChatEvent: ChatEventSchema,
  TickEvent: TickEventSchema,
  ShutdownEvent: ShutdownEventSchema,
};

export const PROTOCOL_VERSION = 3 as const;
```

注意这里还定义了 `PROTOCOL_VERSION = 3` ——这是协议版本号，在客户端连接握手时用于版本协商。

生成的 `dist/protocol.schema.json` 文件结构如下：

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://openclaw.ai/protocol.schema.json",
  "title": "OpenClaw Gateway Protocol",
  "oneOf": [
    { "$ref": "#/definitions/RequestFrame" },
    { "$ref": "#/definitions/ResponseFrame" },
    { "$ref": "#/definitions/EventFrame" }
  ],
  "definitions": {
    "ConnectParams": { "type": "object", "properties": { ... } },
    "HelloOk": { "type": "object", "properties": { ... } },
    "RequestFrame": { "type": "object", "properties": { ... } },
    // ... 80+ 个定义
  }
}
```

12.2.3 从 JSON Schema 生成 Swift 模型

OpenClaw 不仅有 TypeScript 客户端，还有 macOS 和 iOS 原生应用（使用 Swift 编写）。为了保证 Swift 端与 TypeScript 端使用完全相同的协议定义，OpenClaw 提供了一个 Swift 代码生成脚本。

`scripts/protocol-gen-swift.ts` 的核心逻辑是：遍历 `ProtocolSchemas`，将每个 JSON Schema 对象转换为 Swift 的 `struct` 定义：

```
// scripts/protocol-gen-swift.ts（核心生成逻辑，简化展示）

// 1. 生成 ErrorCode 枚举
const header = `
import Foundation

public let GATEWAY_PROTOCOL_VERSION = ${PROTOCOL_VERSION}

public enum ErrorCode: String, Codable, Sendable {
    case notLinked = "NOT_LINKED"
    case notPaired = "NOT_PAIRED"
    case agentTimeout = "AGENT_TIMEOUT"
    case invalidRequest = "INVALID_REQUEST"
    case unavailable = "UNAVAILABLE"
}
`;

// 2. 将 JSON Schema 的 type 映射为 Swift 类型
function swiftType(schema: JsonSchema, required: boolean): string {
  if (schema.type === "string") return required ? "String" : "String?";
  if (schema.type === "integer") return required ? "Int" : "Int?";
  if (schema.type === "boolean") return required ? "Bool" : "Bool?";
  if (schema.type === "array") return `[${swiftType(schema.items, true)}]`;
  return required ? "AnyCodable" : "AnyCodable?";
}

// 3. 为每个 Schema 生成一个 Swift struct
function emitStruct(name: string, schema: JsonSchema): string {
  // 生成属性声明和 CodingKeys
  return `public struct ${name}: Codable, Sendable { ... }`;
}
```

生成的 Swift 代码被写入两个位置：

```
const outPaths = [
  // macOS 应用
  "apps/macos/Sources/OpenClawProtocol/GatewayModels.swift",
  // 跨平台共享库
  "apps/shared/OpenClawKit/Sources/OpenClawProtocol/GatewayModels.swift",
];
```

以 `ConnectParams` 为例，生成的 Swift 代码大致如下：

```
// Generated by scripts/protocol-gen-swift.ts — do not edit by hand
import Foundation

public struct ConnectParams: Codable, Sendable {
    public let minProtocol: Int
    public let maxProtocol: Int
    public let client: Client     // 嵌套 struct
    public let caps: [String]?
    public let commands: [String]?
    public let permissions: [String: Bool]?
    public let device: Device?    // 嵌套 struct
    public let auth: Auth?        // 嵌套 struct

    // ... init 和 CodingKeys 省略
}
```

GatewayFrame 联合类型的特殊处理

Swift 没有像 TypeScript 那样的联合类型（Union Type），但有枚举（enum）可以实现类似效果。 `GatewayFrameSchema` 被特殊处理为一个 Swift 枚举：

```swift
public enum GatewayFrame: Codable, Sendable {
    case req(RequestFrame)
    case res(ResponseFrame)
    case event(EventFrame)
    case unknown(type: String, raw: [String: AnyCodable])

    public init(from decoder: Decoder) throws {
        let typeContainer = try decoder.container(keyedBy: CodingKeys.self)
        let type = try typeContainer.decode(String.self, forKey: .type)
        switch type {
        case "req":   self = .req(try RequestFrame(from: decoder))
        case "res":   self = .res(try ResponseFrame(from: decoder))
        case "event": self = .event(try EventFrame(from: decoder))
        default:
            let raw = try decoder.singleValueContainer()
                .decode([String: AnyCodable].self)
            self = .unknown(type: type, raw: raw)
        }
    }
}
```

注意 `.unknown` 分支——当收到协议版本中未定义的帧类型时，不会崩溃，而是存为一个原始字典。这是一种**前向兼容**（Forward Compatibility）设计：旧版客户端可以安全忽略新版服务器引入的新帧类型。

12.2.4 协议一致性校验

当 TypeBox 定义、JSON Schema 文件和 Swift 模型这三者独立存在时，它们可能因为开发者忘记运行代码生成而不一致。OpenClaw 通过 `pnpm protocol:check` 命令来防止这种情况。

这个检查的基本流程是：

1. 重新运行 `protocol-gen.ts`，生成最新的 JSON Schema
2. 将新生成的 JSON Schema 与仓库中已提交的版本进行**逐字节比较**
3. 如果不一致，报告差异并要求开发者重新运行代码生成

这个检查通常集成在 CI（持续集成）流水线中，确保每次代码提交时协议定义都是同步的。

完整的类型流转闭环

用一张图总结 OpenClaw 协议类型系统的完整流转：

```
┌──────────────────────────────────────────────────────────────┐
│                    TypeBox Schema 定义                        │
│          src/gateway/protocol/schema/*.ts                      │
│                                                                │
│  Type.Object({ message: Type.String(), ... })                  │
└──────────────────────────────────────────────────────────────┘
                        │                    │
                        ▼                    ▼
              ┌──────────────┐     ┌──────────────────┐
              │ Static<typeof>│     │   AJV.compile()   │
              │ 编译时类型提取  │     │ 运行时校验函数编译  │
              └──────────────┘     └──────────────────┘
                        │                    │
                        ▼                    ▼
              ┌──────────────┐     ┌──────────────────┐
              │ TypeScript 类型 │     │  校验函数          │
              │ ConnectParams  │     │ validateConnect()  │
              │ RequestFrame   │     │ validateRequest()  │
              └──────────────┘     └──────────────────┘
                        │
                        ▼
              ┌──────────────────────┐
              │   protocol-gen.ts     │
              │   导出 JSON Schema 文件  │
              └──────────────────────┘
                        │
                        ▼
              ┌──────────────────────┐
              │   protocol-gen-swift.ts  │
              │   生成 Swift Codable 模型  │
              └──────────────────────┘
                        │
                        ▼
              ┌──────────────────────┐
              │   GatewayModels.swift    │
              │   macOS / iOS 原生客户端   │
              └──────────────────────┘
```

这个流转闭环保证了：

- **TypeScript 服务端** 和 **TypeScript 客户端** 共享编译时类型和运行时校验
- **Swift 客户端** 使用自动生成的模型，与服务端定义保持同步
- **CI 检查** 防止任何不一致溜进代码库

这种"单一真相源"（Single Source of Truth）的设计，在协议演进频繁的项目中非常重要。添加一个新的方法参数，只需修改一个 TypeBox 定义文件，然后运行代码生成脚本即可——TypeScript 类型、运行时校验、JSON Schema 和 Swift 模型会自动保持同步。

本节小结

OpenClaw 的 TypeBox 类型系统体现了**类型驱动开发**（Type-Driven Development）的理念：

1. **单一定义，多端复用**：TypeBox Schema 同时提供编译时类型和运行时校验，消除了手动同步的负担
2. **代码生成桥接多语言**：通过 `protocol-gen.ts` 和 `protocol-gen-swift.ts`，同一份协议定义被翻译为 JSON Schema 和 Swift 模型
3. **CI 守护一致性**：`protocol:check` 命令确保生成产物与源定义始终同步
4. **前向兼容设计**：Swift 的 `GatewayFrame` 枚举中的 `.unknown` 分支确保旧客户端能安全处理新帧类型

在下一节中，我们将从 Schema 定义层面下沉到实际的方法和事件层面，看看 Gateway 提供了哪些核心操作能力。